

Problem Roadmap

A progressive 25-problem journey from beginner OOP exercises to senior-level design challenges. Each problem includes the interview prompt, a base spec, and variations that stress-test your design.

How to use this roadmap

Problems are grouped into five tiers of increasing difficulty. Each entry follows the same shape: a one-line interview prompt, the core requirements you should aim to satisfy in your first pass (the *base*), and a set of *variations* that push your design further.

The base version teaches you to model the domain. The variations teach you whether your model is any good. Most of the actual learning happens in the variations — if a small change forces you to rewrite half your classes, that's the lesson.

The recommended loop for each problem

Treat every problem like a real interview. Resist the urge to open your IDE first. Spend time on the spec before any code:

- | | | |
|---------------------------|---------|------------------------------|
| 1. Clarify requirements | ~5 min | (functional + out-of-scope) |
| 2. Identify entities | ~3 min | (nouns become classes) |
| 3. Class design | ~15 min | (state + behavior, top-down) |
| 4. Implement core methods | ~10 min | (happy path, then edges) |
| 5. Verify with a scenario | ~2 min | (trace through tick by tick) |
| 6. Add a variation | | (this is where you grow) |

Once the base is working, pick a single variation and add it. Notice what stays clean and what you have to tear down. Over time you start designing for change without thinking about it.

TIER 1

Foundations

Warm-up problems with small surface area. Focus on clean OOP, single-responsibility classes, and one or two patterns per problem — no more.

1) Tic-Tac-Toe

“Design a two-player tic-tac-toe game on a 3x3 grid.”

Base

Two players alternate placing X and O on a 3x3 grid. A player wins by completing a row, column, or diagonal. The game ends in a draw if all cells are filled with no winner. Invalid moves (occupied cells, moves after the game ends) must be rejected. Provide a way to query current state and reset.

Variations to explore

- Generalize to N×N board with K-in-a-row to win
- Three or more players with distinct symbols
- AI opponent (Strategy pattern: random bot, minimax bot)
- Undo/redo with a move history stack (Memento or Command)
- Decouple game logic from any rendering layer

2) Snake & Ladder

“Design a snake-and-ladder game for multiple players.”

Base

Standard 10x10 board with snakes and ladders. Players take turns rolling a die and moving their token. Landing on a snake's head sends you down; landing on a ladder's foot sends you up. First to the last cell wins.

Variations to explore

- Support multiple dice (sum of N dice per turn)
- Multiple snakes biting at the same cell
- Crooked dice using Strategy pattern (always-even, biased, etc.)
- Tournament mode: track scores across N games
- Configurable board size and snake/ladder positions from input

3) Vending Machine

“Design a vending machine that dispenses items based on user selection and payment.”

Base

User selects an item, inserts payment, and the machine dispenses the item with correct change. Track inventory per slot. Reject selections when out of stock or payment is insufficient.

Variations to explore

- Multiple payment modes (coin, card, UPI) via Strategy
- State pattern for Idle / HasMoney / Dispensing / OutOfStock
- Refund flow when an item becomes unavailable mid-transaction
- Admin role for restocking and price changes
- Multi-item purchase in a single transaction

4) ATM Machine

“Design an ATM that supports authentication, withdrawal, deposit, and balance inquiry.”

Base

User inserts card, enters PIN, and performs operations on their account. ATM must validate the PIN, enforce account balance constraints, and dispense cash from available denominations. Reject invalid PINs after a configurable number of attempts.

Variations to explore

- Multiple account types (savings, current) with different rules
- Daily withdrawal limits per account
- Cash dispensing with multiple note denominations (greedy algorithm)
- State pattern for the card-insertion -> PIN -> operation flow
- Mini-statement and transaction history

TIER 2

Core Interview Problems

These show up the most. Master them — they cover most patterns you'll encounter in real LLD interviews.

5) Parking Lot

“Design a parking lot system where vehicles are assigned to spots as they arrive.”

Base

Vehicles arrive and are assigned an available spot that fits their size. On exit, fees are calculated based on duration and vehicle type. Support cars, bikes, and trucks with differently-sized spots. Track occupancy and reject vehicles when full.

Variations to explore

- Multiple floors with multiple entrances and exits
- Reserved spots for electric vehicles with charging
- Reservation system with future time slots
- Valet parking flow (separate intake)
- Dynamic pricing (peak hours, weekend surcharge) via Strategy
- Chain of parking lots with central management

6) Library Management System

“Design a system to manage a library's books and members.”

Base

Members can search the catalog, check out books, and return them. Track due dates and compute fines for late returns. Support multiple copies of the same title.

Variations to explore

- Reservations and waitlists for popular books
- Different member tiers (student, faculty, guest) with different rules
- Fine calculation as a Strategy (flat, escalating, capped)
- Physical books vs e-books in the same catalog
- Lost-book replacement workflow

7) Splitwise / Expense Sharing

“Design a system where users can record shared expenses and settle balances.”

Base

Users add expenses and split them among participants (equally, exactly, or by percentage). The system maintains a running balance between every pair of users and shows who owes whom.

Variations to explore

- Debt simplification: minimize the number of transactions to settle a group
- Group expenses (trip, roommates) with isolated balance sheets
- Settle-up flow with payment integration
- Multi-currency support with exchange-rate conversion
- Recurring expenses (rent, subscriptions)

8) Logger / Logging Framework

“Design a logging library that supports multiple severity levels and output sinks.”

Base

Applications can log messages at INFO, DEBUG, WARN, or ERROR level. The logger routes messages to one or more sinks (console, file). A configurable threshold controls which levels are emitted.

Variations to explore

- Chain of Responsibility for level-based filtering
- Async logging with a buffer/queue and a background flusher
- Log rotation by size or time
- Structured logging (JSON output with context fields)
- Thread-safe singleton initialization

9) Stack Overflow

“Design a Q&A platform where users post questions and receive answers from others.”

Base

Users post questions, others post answers, and both can be upvoted or downvoted. Each user accumulates reputation based on votes. Search and tag-based discovery let users find relevant questions.

Variations to explore

- Tag system with tag-based feeds and filtering
- Reputation tiers that unlock privileges (comment, edit, moderate)
- Comments on both questions and answers
- Bounties on unanswered questions
- Moderator workflow with permission management

TIER 3

System-Like Problems

Richer domain models with multiple interacting components. You start thinking about subsystems and lifecycles, not just classes.

10) Elevator System

“Design an elevator control system for a multi-floor building.”

Base

An elevator responds to floor requests from inside the car and from each floor's call buttons. It must serve requests in some sensible order, open doors at the correct floor, and remain idle when there are no pending requests.

Variations to explore

- Multiple elevators with a dispatch strategy (nearest car, load balancing)
- Scheduling algorithms (SCAN, LOOK)
- Priority requests (VIP, freight, emergency)
- Maintenance mode that takes a car out of service
- Capacity and weight limits with overload handling

11) Movie Booking System (BookMyShow)

“Design a system for users to browse shows and book seats at theatres.”

Base

Theatres host multiple shows; each show has a fixed set of seats. Users search by city, movie, and time, select seats, and pay. The system prevents two users from booking the same seat for the same show.

Variations to explore

- Seat locking with timeout (concurrency control)
- Payment integration with multiple gateways
- Dynamic pricing for premium seats and weekend surcharge
- Cancellation and refund policies
- Personalized recommendations based on user history

12) Cab Booking (Uber/Ola)

“Design a ride-hailing system that matches riders with nearby drivers.”

Base

Riders request a ride from their location to a destination. The system finds a nearby available driver and creates a ride. The ride proceeds through a defined lifecycle (requested -> assigned -> in-progress -> completed) with fare calculation at the end.

Variations to explore

- Surge pricing via Strategy pattern
- Driver matching algorithms (nearest, highest-rated, ETA-optimized)
- Ride pooling / shared rides
- Two-way ratings (driver rates rider, vice versa)
- Cancellation policies and fees

13) Food Delivery (Swiggy/Zomato)

“Design a food-delivery platform that connects users, restaurants, and delivery agents.”

Base

Users browse restaurants, build a cart, place an order, and a delivery agent picks it up and delivers it. The order moves through placed -> accepted -> prepared -> picked-up -> delivered, with status visible to all parties.

Variations to explore

- Delivery agent assignment algorithm
- Real-time order tracking using Observer pattern
- Restaurant ratings and reviews
- Offers, coupons, and loyalty programs
- Multiple payment methods including cash on delivery

14) Hotel Booking System

“Design a hotel booking system supporting search, availability, and reservations.”

Base

Users search hotels by location and date range, view available rooms, and make a reservation. The system must prevent double-booking the same room for overlapping dates.

Variations to explore

- Multiple room types (standard, deluxe, suite) with distinct pricing
- Availability calendar supporting date-range queries efficiently
- Dynamic pricing based on demand and season
- Group bookings and corporate accounts
- Loyalty program with reward tiers

15) Online Shopping (Amazon-lite)

“Design an e-commerce backend that supports browsing, cart, checkout, and order fulfillment.”

Base

Users browse a product catalog, add items to a cart, and check out. Orders are created, payment is captured, and inventory is decremented. Users can track their order status.

Variations to explore

- Inventory across multiple warehouses with allocation logic
- Product recommendations (collaborative or content-based)
- Reviews and ratings with moderation
- Return and refund flow
- Stock-back-in-stock alerts using Observer pattern

TIER 4

Concurrency-Heavy & Advanced

Problems that force you to reason about thread safety, locks, and trade-offs. Standard territory for senior interview loops.

16) Cache (LRU / LFU)

“Design an in-memory cache with $O(1)$ get and put.”

Base

Build a fixed-capacity cache. On get, return the value if present; on put, evict the least-recently-used entry when over capacity. Both operations must be $O(1)$ average.

Variations to explore

- LFU (least-frequently-used) eviction policy
- TTL-based eviction (entries expire after a duration)
- Thread-safe implementation with locks
- Write-through vs write-back semantics
- Pluggable eviction strategies (Strategy pattern)

17) Rate Limiter

“Design a rate limiter for an API endpoint.”

Base

Given a key (e.g. user ID or IP), determine whether a request should be allowed or rejected based on a configured limit (e.g. 100 requests per minute). Must be efficient and accurate at the boundaries.

Variations to explore

- Token bucket vs leaky bucket vs sliding window comparison
- Distributed rate limiting (Redis-based) concepts
- Per-user vs per-API vs per-IP limits
- Configurable limits per endpoint with hot reload
- Burst handling and graceful degradation

18) Concurrent Hash Map

“Design a thread-safe hash map.”

Base

Support concurrent get, put, and remove operations from many threads. Reads should be fast; writes should be safe. Avoid blocking the entire map for every operation.

Variations to explore

- Segmented locking for higher concurrency
- Read-write locks for read-heavy workloads
- Lock-free reads using atomic operations
- Resize logic that doesn't block all readers

19) Task / Job Scheduler

“Design a system that executes jobs at specified times.”

Base

Users submit jobs along with a time at which to run. The scheduler dispatches jobs to workers at or after the scheduled time. Jobs may be one-shot or recurring.

Variations to explore

- Cron-like recurring expressions
- Priority queue for ordering
- Distributed scheduling across worker pools
- Retry policies with exponential backoff
- Job dependencies (DAG) and topological execution

20) Notification System

“Design a system that sends notifications across multiple channels.”

Base

Given a notification payload (recipient, message, channels), deliver it via email, SMS, or push. Apply user preferences, retry on failure, and avoid sending duplicates.

Variations to explore

- User preferences (channel, frequency, do-not-disturb hours)
- Batching of similar notifications into digests
- Retry on delivery failure with backoff
- Template management with parameter substitution
- Scheduled (future-time) notifications

TIER 5

Stretch & Senior-Level

Problems that combine multiple subsystems, complex domain rules, and serious design trade-offs. Tackle once Tiers 1-4 feel routine.

21) Chess Game

“Design a two-player chess game with full rules.”

Base

Standard 8x8 board with the six standard piece types. Players alternate moves; each piece has its own movement rules. The game ends in checkmate, stalemate, or draw.

Variations to explore

- Per-piece movement as Strategy
- Undo/redo via Memento
- Check, checkmate, and stalemate detection
- Special moves (castling, en passant, pawn promotion)
- Save and load games (serialization)

22) Distributed Key-Value Store

“Design a distributed key-value store that scales horizontally.”

Base

Support get, put, and delete distributed across multiple nodes. Data must remain available even when individual nodes fail. The client should not need to know which node owns which key.

Variations to explore

- Consistency models (strong, eventual)
- Replication strategies (leader-follower, multi-leader)
- Conflict resolution (last-write-wins, vector clocks)
- Consistent hashing for partitioning
- Read repair and anti-entropy

23) URL Shortener

“Design a service that converts long URLs into short ones and redirects on lookup.”

Base

Given a long URL, generate a unique short URL. When a user visits the short URL, redirect to the original. Track basic analytics on each redirect.

Variations to explore

- Encoding strategies (base62, hash-based)
- Collision handling for hashed schemes
- Click analytics (count, geography, referrer)
- Custom (vanity) URLs
- Link expiry and deletion

24) Calendar / Meeting Scheduler

“Design a calendar system that supports events, invites, and conflict detection.”

Base

Users create events with attendees and time ranges. The system invites attendees and tracks responses. It must detect conflicts and offer to find common free slots.

Variations to explore

- Find common free slots across many attendees
- Recurring meetings (daily, weekly, monthly, custom)
- Time zone handling
- Conflict detection and resolution
- Meeting room booking integration

25) Stock Exchange / Trading System

“Design the order matching engine of a stock exchange.”

Base

Users submit buy and sell orders for a security at a given price and quantity. The matching engine matches compatible orders, executes trades, and updates the order book. Must be deterministic and fast.

Variations to explore

- Multiple order types (market, limit, stop-loss)
- Price-time priority order book using priority queues
- Real-time price updates via Observer
- Trade settlement workflow
- Multiple symbols / multiple exchanges

A few habits that compound

Don't skip the design phase

Spend 10-15 minutes on a doc or whiteboard before opening your IDE. List requirements, identify entities, sketch a rough class diagram. Real interviews work this way, and your design quality at home will mirror what you produce under pressure.

Variations matter more than the base

Anyone can build a base version that works. Good design is about whether your code can absorb a new requirement without a rewrite. If you only ever build base versions, you're skipping the part that makes you a better designer.

Don't force patterns

If a problem doesn't naturally call for Observer, don't bolt it on. Patterns are a vocabulary for describing solutions you'd arrive at anyway, not a checklist to tick off. Forced patterns are a stronger negative signal than missing ones.

Ask: where does this responsibility belong?

The Single Responsibility Principle is the most useful principle in LLD interviews. If a class is doing two things, split it. Keep rules with the entity that owns the relevant state — this is what 'tell, don't ask' really means.

Code end-to-end, even if minimal

A working tic-tac-toe with a clean design beats a half-built parking lot every time. Finishing teaches you about state transitions, edge cases, and integration in a way that starting never does.

Your first five

If you want a starter path: Tic-Tac-Toe → Vending Machine → Splitwise → Logger → Elevator. After these, you'll have touched most major patterns at least once and built the muscle for everything that follows.